CPRE 4870/5870 - Final Project

Cameron Gilbertson, Tyler Bibus, Reza Choudhury

Date: 12/6/2025

Table of Contents

Table of Contents 2
Introduction 3
Motivations 3
Background 3
Baseline Goal 3
Stretch Goal 1 (Profiling Different Network Sizes) 4
Stretch Goal 2 (Full Transformer HW Accelerator) 4
Stretch Goal 3 (C++ Optimizations) 4
Literature Search 5
Foundations for a ViT model 5
ViT Training Strategies 6
Differences between CNNs and ViTs 6
Quantization and Hardware Acceleration 7
Other Mechanisms/Models Relevant to Our Project 8
Design Approach 9
Overall Project Approach 9
Software 9
Hardware 10
Results 13
Baseline Goal 13
Python Implementation 13
C++ Implementation 15
Hardware Integration 17
Stretch Goal 1 (Profiling Different Network Sizes) 19
Baseline Mode Size Results 19
Half-Model Size Results 19
Double-Model Size Results 20
10x-Model Size Results 21
Overall Results 21
Stretch Goal 2 (Full Transformer HW Accelerator) 22
Stretch Goal 3 (C++ Optimizations CPU) 23
Naive 23
Multi-Threading 23
Tiling 23
Attention Streaming 24
SIMD (ARM NEON) 24
Challenges 26
Software 26
Hardware 26

Future Plans **28**
Full Transformer Hardware Accelerator 28
Hardware Accelerator in FP32 28
Works Cited **29**

Introduction

Motivations

In this course we explored the basics of a CNN through the Lab 1, 2, and 3. These labs painstakingly taught us the fundamentals through many experiments. Then the following labs (4, 5, 6), work on building upon the fundamentals and towards more optimized implementations. Through these labs we learn lots about CNNs and machine learning and we wanted a similar experience to these labs but on a new modern topic.

The new topic of machine learning we chose to explore was transformers. Transformers are the more modern application of machine learning used in popular large language models (Chat-GPT, Gemini). These piqued our interest due to their scalability with increasing parameter sizes.

Background

Transformers are a relatively novel approach to machine learning, becoming a defined architecture in 2017 in the paper *Attention Is All You Need*. Transformers are different from CNNs in that they calculate in a stochastic manner using attention instead of just inputs and weights like a CNN model.

Attention is a process where a Query, Key, and Value are multiplied and summed across matrices simultaneously, allowing the model to effectively "see" the entire input at once. This is different from CNNs which are limited by their filter sizes; however this does cause transformers to lose inductive bias, which is made up for with positional encoding. To get a better understanding of Transformer architecture, I would recommend reading some of the summaries, as well as the papers themselves in the literature search section.

Baseline Goal

The baseline goal for this project was to complete the first three labs (1, 2, and 3) but using transformers instead of our given CNN model. The baseline can be broken down into three stages:

- Python Implementation (Keras ViT Baseline) (Lab 1)
- C++ Implementation based on Python Framework (Lab 2)

- Basic HW accelerator (Similar to Lab 3 FIFO MACs) (Lab 3)

The reason we are doing it this way similar to the labs is because each portion builds on each other. So once we have the Python implementation we will have outputs to compare for our C++ implementation. Then once we have the C++ implementation we will integrate the hardware using a FP32 MAC and an AXI-4 FIFO unit.

Stretch Goal 1 (Profiling Different Network Sizes)

The first stretch goal was to explore the scalability of the transformers in our python implementation. We trained four models in total based on different amounts of parameters(½ size, full-size, 2 size, 10 size) and compared each of their accuracies. Then based on our results suggesting a model we would use moving forward.

Stretch Goal 2 (Full Transformer HW Accelerator)

The second stretch goal was to build a HW Accelerator similar to the Lab 6 HW accelerator. We were not able to implement this because of time constraints but we do have a theoretical implementation that we believe would work.

Stretch Goal 3 (C++ Optimizations)

The third stretch goal was not originally in our proposal but after completing Lab 5 and seeing those results and speedup we were interested in applying those techniques to our transformer model. We also believed that implementing these techniques would be feasible in a reasonable amount of time because we learned each of these techniques in Lab 5.

Literature Search

I have found these papers to be both related and informative. These papers have been broken down into sections for how they have related/assisted in the development of this project. There is a large amount of overlap between papers, which is why they are largely grouped the way they are.

Foundations for a ViT model

Attention Is All You Need (Vaswani et al., 2017)
Attention Is All You Need is the paper that coined the Transformer architecture. They focus largely on translation tasks, proving that with relatively low training and low parameterization their new model – using exclusively attention – was able to outperform many other ML models. They created the strategies and concepts that are still in use at scale today, with their general architecture being a game changer in ML. They largely based their designs off previous architectures like Recurrent Neural Networks and Convolutional Neural Networks, but ended up creating the distinct design. Multiple headed designs improve pattern recognition and reduce them into rules to follow. These rules can be like grammatical rules and patterns in languages, or in ViT rules like a dog's tail is at its rear. This paper truly is disruptive in its implications for machine learning.

An Image is Worth 16x16 Words: Transformers for ImageRecognition at Scale (Dosovitskiy et al, 2020)
This paper is similar in its importance to *Attention Is All You Need* for ViTs. This paper created the strategies still used today for ViT models, specifically patchification. They also find that larger datasets can fix issues that appear with the lack of inductive bias, even though they inject positional encoding more training data is better for the model. In this paper they try to stick to the original transformer (Vaswani et al., 2017) as much as possible, focusing largely on scalability. They started by pretraining on large datasets, before fine-tuning the model down to smaller tasks. We implemented this, as we use the *timm* model library, allowing us to tune to our smaller dataset. This was critical as if we had to train from scratch, the 100,000 training set images would likely be insufficient. In the paper they find that their largest model was able to beat BiT-L (ResNet) in accuracy with much fewer TPU v3-core-days (effectively a ¼ of the pre-training cost). They also found that the larger the dataset, the better their model performed – especially for top-1 accuracy – with diminishing returns. They find that their ViT model scales much better,

dominating the ResNets on performance/compute, but being slightly edged out from a hybrid model – hybrid models have a CNN layer that feeds in some feature maps which can be beneficial for the Transformer network.

A Comprehensive Study of Vision Transformers in Image Classification Tasks (Khalil et al., 2023)
This paper focuses on the effects of Transformers on the Image Classification research area. They look over traditional ViT models, Swin Transformers, DeiT, CaiT, iGPT, and compare their performance. Original ViT models focused on just using Transformers with pretraining, while iGPT, CLIP, Model Soups, and others focused largely on improving generalization without needing retraining, improving flexibility. This paper was very helpful in the way that it effectively walks through the evolution of ViT models from concepts like attention to model soups. This paper is very relevant and a great read to get a better grasp on Transfomers in Vision tasks and their implications. In terms of actual application, this paper helped us get a better grasp of how our own model functioned by contrasting it with other applications. The Swin Transformer is a very interesting approach, and is also covered in a later paper that will be discussed.

ViT Training Strategies

Training data-efficient image transformers & distillation through attention (Touvron et al., 2021)
This paper focuses on training one Imagenet1k only. They introduce a concept of a "teacher-student" strategy, which improves model learning. What is particularly interesting about this paper is that it proposes introducing a new distillation token in addition to the class token. This, combined with soft and hard distillation allows their model to learn from the output of the teacher. They are able to use a different model, like a convolutional model as the teacher for the transformer model. While we did not use this strategy when developing our project, it is a very interesting read as its implications on training models cannot be understated.

Differences between CNNs and ViTs

Do Vision Transformers See Like Convolutional Neural Networks? (Raghu et al., 2021)
This paper focuses on how ViT models actually find their classification compared to CNNs. CNN models find features layer by layer, distilling down these features before

finally doing fully connected and classifying the model. This paper finds that ViT models are much different, effectively seeing a more uniform representation across lower and higher layers. ViT also incorporates global information compared to CNN models. The part on receptive fields is particularly interesting, as ViTs and CNNs receptive fields work in almost opposite ways. A CNN model will start in a very small block before slowly expanding across blocks, while ViT models start more global before focusing down. While this paper didn't give any specific math, or help directly with our software/hardware implementation, the information given on the differences were incredibly insightful. This contrast between CNNs and ViTs is what our project was aiming for, and this paper exemplifies these differences in "vision" perfectly.

A ConvNet for the 2020s (Liu et al., 2022)

This paper focuses on improving a CNN, using strategies learned from ViT models. One thing that I would like to point out is that our model uses GELU while our CNN model uses ReLU. This is something that they address in this paper – which is a large reason why I picked it – and it found that accuracy remained unchanged with the "smoother" activation function. This is a relatively large revelation, as if this result remained for our model, we would see that Transformers contributed more to our overall accuracy improvement, rather than improvements like GELU. While our baseline CNN model does not use BatchNormalization or LinearNormalization, this is another change that improves accuracy. Perhaps implementing LinearNormalization – mostly as an overfitting prevention strategy – could be very beneficial to our CNN model. Exploring this possibility could be a good project. They find that after making many of these changes to the CNN they were able to outperform the Swin Transformer, proving that CNNs were, in fact, still a competitive option.

Quantization and Hardware Acceleration

Model Quantization and Hardware Acceleration for Vision Transformers: A Comprehensive Survey (Du et al., 2024)

This paper served as a basis for our stretch goal 2 – a full ViT hardware accelerator. While we didn't end up being able to implement the full accelerator, this paper is very important for that purpose. This paper focuses on quantizing in addition to HW acceleration for ViT models. They cover baseline quantization – which would have been helpful for lab 3 – as well as diving into ViT quantization specifically. Most challenges to ViT quantization that does not necessarily exist in CNNs comes from quantizing after operations like Softmax, LayerNormalization, and GELU. Most quantization methods they cover do not require retraining and can be done post quantization. The optimal

quantization method we would likely go with is the method the TSPTQ-ViT utilizes, as it does quantization for all three operations that we utilize (Softmax, LayerNorm, GELU). It also does not require retraining and maintains the best accuracy when quantized to 8-bit which would be our targeted quantization level. For hardware strategies we would use, their strategy for softmax by using a shift arithmetic function is exactly what we would need and use. They also apply shifting as a way to save computations in GELU. Combining these with our mac units and Lab 6 accelerator would be a probable way for us to implement a hardware accelerator.

Other Mechanisms/Models Relevant to Our Project

Swin Transformer: Hierarchical Vision Transformer using Shifted Windows (Liu et al., 2021)

This paper focuses on a ViT model that uses a shifted window approach for computing self-attention. As discussed in papers mentioned earlier this approach is very interesting. By using shifted windows they are able to greatly reduce latency compared to sliding windows. These shifted windows are non-overlapping patches, with a hierarchical approach where each layer has different shifted windows. This approach replaces the multi-head self attention. The first module uses a standard window partitioning system, but subsequent layers are shifted. This approach allowed them to beat both previous state-of-the-art ViT and CNN models in accuracy. While our ViT model is standard in how it uses a mult-head self-attention rather than shifted windows, these improvements pushed image classification models even higher. An interesting future project could be comparing a shifting window model to our basic ViT model and see how much more accuracy we can achieve.

Gaussian Error Linear Units (GELUs) (Hendrycks et al., 2016)

While this paper might not be quite as exciting as previous papers covered, its importance cannot be understated. It defines a new activation function by combining multiple properties as well as ReLU. The main benefits of GELU over ReLU is that when a result is less than zero, ReLU causes those neurons to die out. This paper's alternative GELU is a lot smoother, while also preventing nearly as many neurons from dying out. They tested GELU on MNIST classification, which is a much simpler dataset than ImageNet. GELU outperformed both ReLU and ELU, which are previous non-linear activation functions. This paper helped us explicitly, as our model from the *timm* library uses GELU. The math laid out, as well as the graphs, were crucial to recreating our Python model manually.

Design Approach

Overall Project Approach

Our overall approach was to implement each portion of the project in similar order to the order of the first three labs. This would give us baselines to implement the next part of the project. Then once we have completed the baseline goal we moved onto our stretch goals. But our project can be broken down into two larger sections each with different approaches.

Software

The approach for our software development of the project was to first find a publicly available model to implement in Python. This gave us a software baseline for our project and helped with understanding how the math behind transformers relates to a real implementation. Along the way we created several debugging and visualization tools and in Python which helped us immensely and provided great visuals for this report and the demo.

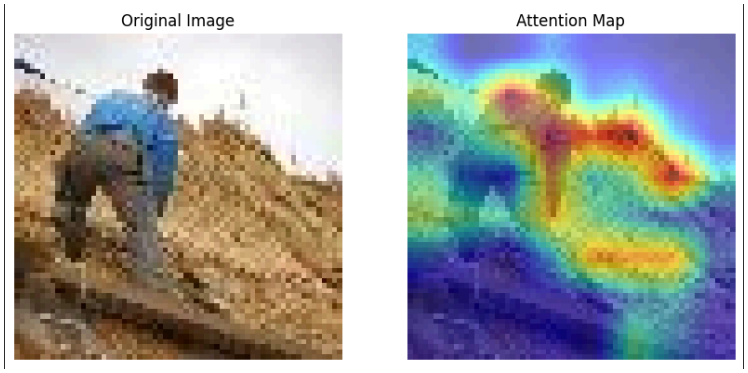


Figure A: *Output Attention Map from Python*

Once we had a working Python implementation we used that to modify the given C++ framework from the labs. Overall we had to add some new layers but most of the backend of the framework remained the same. This was nice because it removed a lot of framework debugging that would have been painful if we had started a new framework from scratch.

Finally, when we were able to run a full inference on the lab computers. We modified the software to integrate with the hardware using an approach to Lab 3.

Hardware

The hardware for our final project was similar to our Lab 3 except our transformer model is in FP32 and not quantized to Int8. This means we had to modify our MAC to work in FP32. We had two main issues we needed to resolve to work with our Lab 3. The first issue is that each FP32 takes a 32 bit word. Our approach: since the FIFO outputs 32 bit words, we need two words before we can multiply and accumulate. So we must wait until we have received two valid words and then continue. Refer to Figure B to see the state machine logic. Another issue was the computational expense of multiplying 32 bit numbers. Our resolution was to use a built-in MAC IP from Vivado inside of our FP32 MAC unit. This MAC is more optimized than just a multiply symbol in behavioral VHDL. Refer to Figure B to see the overview of the MAC IP.

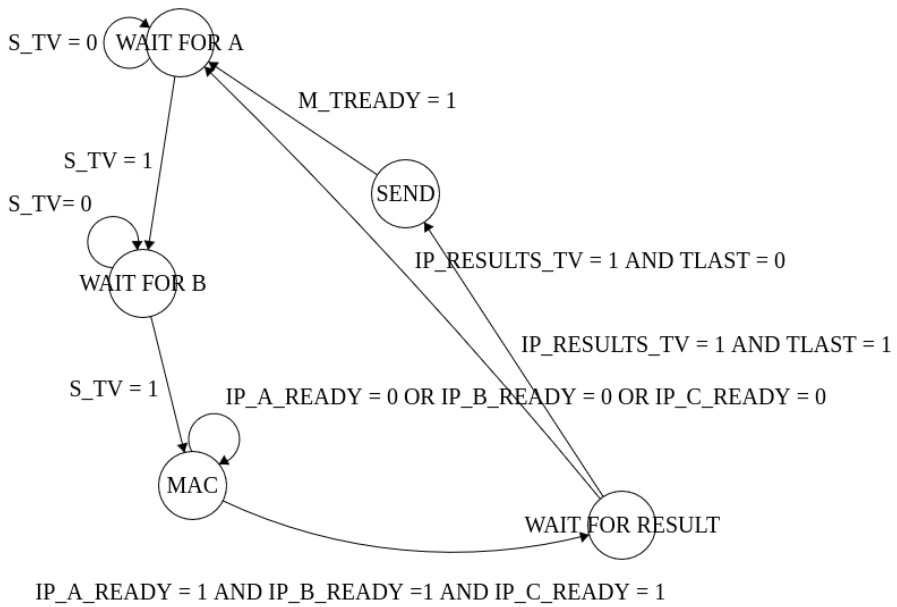


Figure B: *FP32 MAC State Machine*

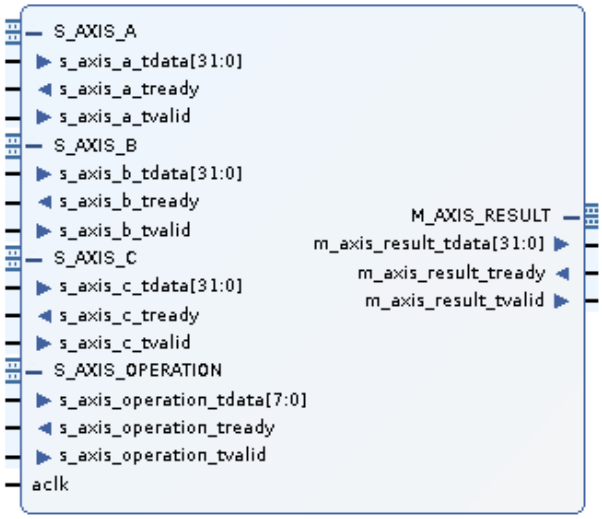


Figure C: *MAC IP from Vivado*

Our approach to test this was to first test the MAC unit in simulation using some custom testbenches. Refer to Figure D to see the test bench results.

Once the MAC was working in simulation, instead of directly implementing the MAC and FIFO unit into the transformer. We thought it would be better to go back to Lab 2 and implement our FP32 CNN in hardware first. This was a great idea because it ironed out all of the issues using the CNN logic we were very familiar with instead of the transformer logic which was still difficult.

After having a working Lab 2 implementation we integrated the hardware into our transformer which proved to be easier since we had our Lab 2 implementation to base it off of. Refer to Figure E and F for our full inference time results for the Lab 2 implementation.

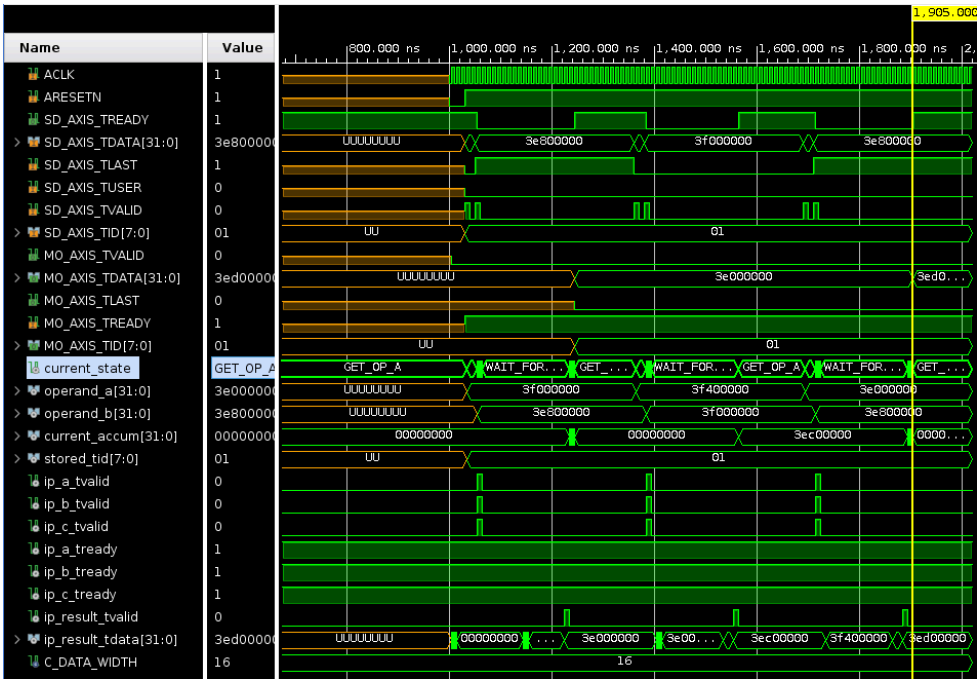


Figure D: *FP32 MAC Test Bench*

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/image_0.bin
xsdb% Timer Full Inference: elapsed=58695.000000ms
xsdb% Opened binary file data/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
```

Figure E: *FP32 MAC CNN Full Inference Time*

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/image_0.bin
xsdb% Timer Full Inference: elapsed=29088.000000ms
xsdb% Opened binary file data/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): True 96.1316%  (0.961316)
```

Figure F: *Int8 MAC CNN Full Inference Time*

Results

Baseline Goal

Python Implementation

Our first goal is to develop a Python implementation of a transformer to do full inference using TinyImageNet. For this we used Pytorch with the timm transformer model. Using this we were able to build and train a model that we could do inference on. Refer to Figure H for examples of our training.

Our baseline model has less parameters than our CNN model we used in the labs. Refer to Figure G for model parameter details. Our goal was to have around the same amount of parameters as our CNN model for a more valid comparison. But our results when comparing the two models were interesting and surprising.

```
==========================================
Total params: 642,728
Trainable params: 642,728
Non-trainable params: 0
Total mult-adds (M): 1.80
==========================================
Input size (MB): 0.05
Forward/backward pass size (MB): 3.69
Params size (MB): 2.55
Estimated Total Size (MB): 6.29
==========================================
```

Figure G: *Baseline Model Size*

```
Ep 6: Train Acc: 23.53% | Val Acc: 24.17% | Val Loss: 3.3327
Epoch 7/30: 100%|██████████| 782/782 [01:00<00:00, 12.88it/s, loss=3.26]
Ep 7: Train Acc: 25.68% | Val Acc: 26.76% | Val Loss: 3.2149
Epoch 8/30: 100%|██████████| 782/782 [01:00<00:00, 12.85it/s, loss=3.13]
Ep 8: Train Acc: 28.04% | Val Acc: 28.13% | Val Loss: 3.1350
Epoch 9/30: 100%|██████████| 782/782 [00:59<00:00, 13.05it/s, loss=3.03]
Ep 9: Train Acc: 29.85% | Val Acc: 29.77% | Val Loss: 3.0726
Epoch 10/30: 100%|██████████| 782/782 [01:00<00:00, 12.93it/s, loss=2.92]
Ep 10: Train Acc: 31.72% | Val Acc: 30.26% | Val Loss: 3.0122
Epoch 11/30: 100%|██████████| 782/782 [01:00<00:00, 12.84it/s, loss=2.84]
Ep 11: Train Acc: 33.36% | Val Acc: 32.23% | Val Loss: 2.9262
Epoch 12/30: 100%|██████████| 782/782 [01:01<00:00, 12.81it/s, loss=2.76]
Ep 12: Train Acc: 34.95% | Val Acc: 33.32% | Val Loss: 2.8629
Epoch 13/30: 100%|██████████| 782/782 [01:01<00:00, 12.64it/s, loss=2.69]
Ep 13: Train Acc: 36.27% | Val Acc: 34.05% | Val Loss: 2.8410
Epoch 14/30: 100%|██████████| 782/782 [01:01<00:00, 12.78it/s, loss=2.62]
Ep 14: Train Acc: 37.54% | Val Acc: 35.07% | Val Loss: 2.7836
Epoch 15/30: 100%|██████████| 782/782 [00:59<00:00, 13.14it/s, loss=2.55]
Ep 15: Train Acc: 38.92% | Val Acc: 35.69% | Val Loss: 2.7642
Epoch 16/30: 100%|██████████| 782/782 [01:00<00:00, 12.98it/s, loss=2.5]
Ep 16: Train Acc: 39.92% | Val Acc: 36.05% | Val Loss: 2.7471
Epoch 17/30: 100%|██████████| 782/782 [01:00<00:00, 13.02it/s, loss=2.44]
Ep 17: Train Acc: 41.14% | Val Acc: 36.83% | Val Loss: 2.7277
Epoch 18/30: 100%|██████████| 782/782 [00:59<00:00, 13.19it/s, loss=2.38]
Ep 18: Train Acc: 42.34% | Val Acc: 37.23% | Val Loss: 2.7162
Epoch 19/30: 100%|██████████| 782/782 [00:59<00:00, 13.05it/s, loss=2.33]
Ep 19: Train Acc: 43.30% | Val Acc: 37.51% | Val Loss: 2.6892
Epoch 20/30: 100%|██████████| 782/782 [00:59<00:00, 13.15it/s, loss=2.28]
Ep 20: Train Acc: 44.47% | Val Acc: 37.88% | Val Loss: 2.6642
Epoch 21/30: 100%|██████████| 782/782 [01:00<00:00, 12.95it/s, loss=2.22]
Ep 21: Train Acc: 45.53% | Val Acc: 38.32% | Val Loss: 2.6687
Epoch 22/30: 100%|██████████| 782/782 [01:00<00:00, 12.94it/s, loss=2.18]
Ep 22: Train Acc: 46.36% | Val Acc: 38.52% | Val Loss: 2.6554
Epoch 23/30: 100%|██████████| 782/782 [01:00<00:00, 12.91it/s, loss=2.14]
Ep 23: Train Acc: 47.12% | Val Acc: 38.84% | Val Loss: 2.6482
Epoch 24/30: 100%|██████████| 782/782 [01:01<00:00, 12.77it/s, loss=2.1]
Ep 24: Train Acc: 48.05% | Val Acc: 39.17% | Val Loss: 2.6346
Epoch 25/30: 100%|██████████| 782/782 [01:01<00:00, 12.81it/s, loss=2.07]
Ep 25: Train Acc: 48.73% | Val Acc: 39.12% | Val Loss: 2.6453
Epoch 26/30: 100%|██████████| 782/782 [00:59<00:00, 13.16it/s, loss=2.04]
Ep 26: Train Acc: 49.31% | Val Acc: 39.39% | Val Loss: 2.6292
Epoch 27/30: 100%|██████████| 782/782 [01:00<00:00, 13.01it/s, loss=2.01]
Ep 27: Train Acc: 49.91% | Val Acc: 39.53% | Val Loss: 2.6291
Epoch 28/30: 100%|██████████| 782/782 [00:59<00:00, 13.21it/s, loss=2]
Ep 28: Train Acc: 50.26% | Val Acc: 39.36% | Val Loss: 2.6294
Epoch 29/30: 100%|██████████| 782/782 [01:00<00:00, 12.99it/s, loss=1.99]
Ep 29: Train Acc: 50.74% | Val Acc: 39.67% | Val Loss: 2.6241
Epoch 30/30: 100%|██████████| 782/782 [00:59<00:00, 13.09it/s, loss=1.98]
Ep 30: Train Acc: 50.64% | Val Acc: 39.62% | Val Loss: 2.6239
```

Figure H: *Python Training Outputs*

The accuracy of our transformer model significantly outperforms our CNN model by around 16% in all categories. Our expectation was there to be an accuracy dropoff for our transformer model because usually a CNN should outperform a transformer model of similar size. The advantage of transformers is their scalability and to have it outperform our CNN model by so much was very surprising.

Our reasoning for why this occurs is because the transformer model we found is just better than our CNN model. We suspect that our CNN model is out-dated and/or too simple. Our CNN model from the labs has a limited amount of layers to reduce complexity. Which we were happy about when we were completing the labs but that could reduce performance.

```
==============================
VALIDATION RESULTS
==============================
Top-1 Accuracy: 39.62%
Top-5 Accuracy: 65.60%
Top-10 Accuracy: 75.99%
==============================
```

Figure I: *Python Implementation Accuracy*

```
### Calculating Top-1, Top-5, Top-10 Accuracy ###
Using softmax data from: img_data/8bitSoftmax
2025-10-29 23:47:44.354295: W tensorflow/core/kernels
Top-1 Accuracy: 23.70%
Top-5 Accuracy: 49.80%
Top-10 Accuracy: 61.00%
```

Figure J: *CNN 8-bit Model Accuracy from Lab 4*

Overall this was our first major step in the project and it felt good to get a working transformer model to base our C++ implementation on.

C++ Implementation

The next step for our project was to get a working C++ implementation based on our Python implementation. This was going to be an important step for our last part of the project hardware integration because our hardware integration will use the C++ software.

When implementing the C++ our first step was to modify the given Lab 2 framework to fit our transformer model. This involved removing some layers like Convolution and Flatten. Then adding some new layers like PatchEmbed, TransformerBlock, Normalization Layer, and Token Extraction. After adding these files, we modified the ML.cpp file to fit our new model. After creating the shell we implemented each of these layers individually testing as we went until we were able to do inference and get some results.

Our results were very pleasing to see. For a single inference, our model took 15.45ms total. Which is around 6 times faster than our base CNN model C++ Implementation. This was expected because transformers do less computations than CNN of similar sizes.

```
[Info]: --- Running Inference Test ---
Opened binary file data/image_0.bin
[Info]: Input data loaded.
Timer Full Inference: elapsed=15.456000ms
Opened binary file data/image_0_data/layer_12_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
```

Figure J: *Transformer C++ Implementation Inference Time*

```
[Info]: --- Running Inference Test ---
Opened binary file data/image_0.bin
Timer Full Inference: elapsed=88.583000ms
Opened binary file data/image_0_data/layer_11_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
```

Figure K: *CNN C++ Implementation Inference Time*

Our layerwise plot was interesting and tells us lots about how our model works. It makes sense that most of the computations are inside the PatchEmbed and TransformerBlock layers. These layers involve the most amount of MAC operations which is lots of the computation time. But a result to note especially for when we go to the hardware integration section.

Otherwise our other layers involve few computations but are necessary in converting the output of the transformer blocks into usable data.

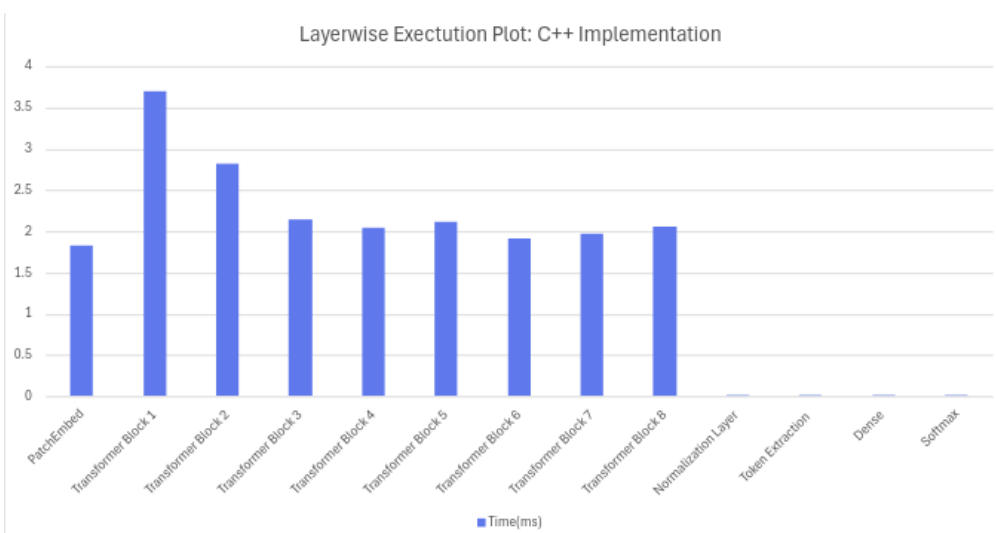


Figure L: *C++ Implementation Layerwise Execution Plot*

The accuracy of our C++ implementation was higher than our Python implementation. This was surprising until we realized that C++ model accuracy measurement was across 1,000 images while our Python model was across the entire validation set, so 10,000 images. This difference in accuracy probably means that our C++ implementation "got lucky" with its 1,000 images that were easier to identify.

```
Loading results from data/inference_outputs...
------------------------------
Processed: 1000 images
Top-1 Accuracy:  44.80%
Top-5 Accuracy:  71.00%
Top-10 Accuracy: 80.70%
------------------------------
```

Figure M: *C++ Implementation Accuracy*

Hardware Integration

Our results for the hardware integration were as expected. Similar to Lab 3, there were significant slow downs compared to the software version due to there only being 1 PE. See Figure L for the total inference time. But overall this shows that the FP32 MAC we created was able to replace all of the MAC operations inside of our transformer model and run them on the ZedBoard.

Obviously the layers with the most slowdown were the ones with the most amount of MAC operations. But interestingly since each of the transformer blocks are the same dimensions so they had the same amount of MAC operations. So unlike the software layerwise plot, all of the transformer blocks layers have almost identical execution time. Another result from the hardware layerwise plot compared to the software layerwise plot was the layer with the largest execution time. For our software layerwise plot, the layer with the longest execution time was transformer block 2. But for our hardware layerwise plot, the layer with the longest execution time was the patchembed layer. There are a couple reasons why this is possible.

One reason is that the patchembed layer has more MAC operations than any other layers. This would explain why patchembed has the largest execution time for hardware. Another reason is that the transformer layers have some other operations that take time

but since MAC operations are cheap on the lab computers the difference in MAC operations does not make too large of an impact on execution time.

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/image_0.bin
xsdb% [Info]: Input data loaded.
xsdb% Timer Full Inference: elapsed=3703.000000ms
xsdb% Opened binary file data/image_0_data/layer_12_output.bin
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
```

Figure N: *Hardware Integrated Inference Time*

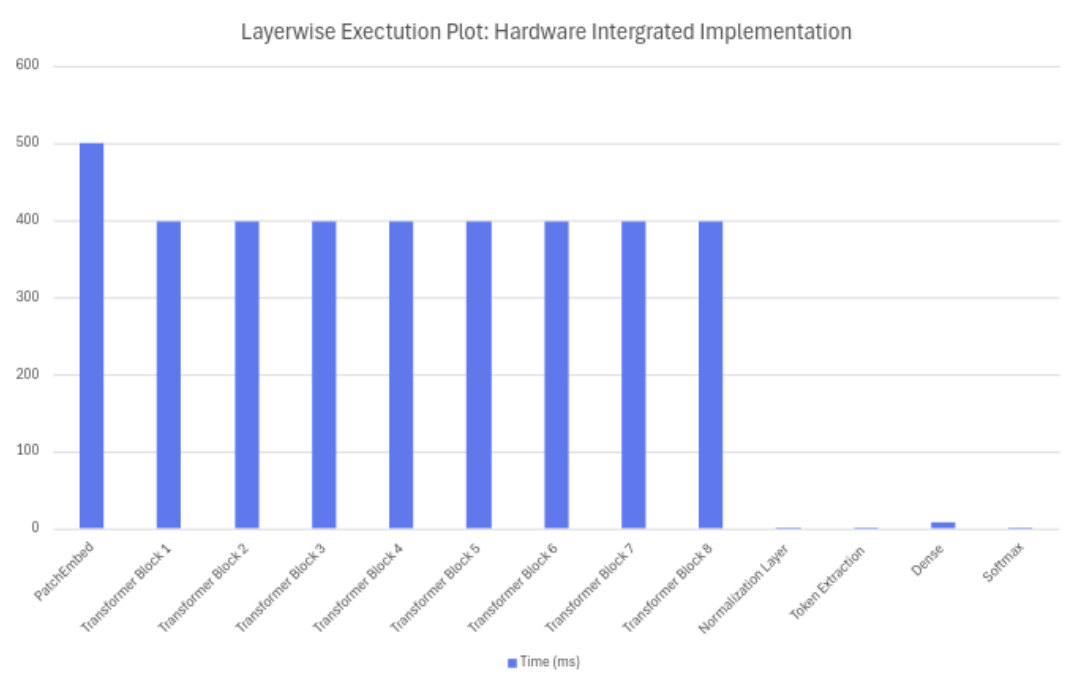


Figure O: *Hardware Integrated Layerwise Execution Plot*

Stretch Goal 1 (Profiling Different Network Sizes)

Baseline Mode Size Results

For our first stretch goal we wanted to see how model size impacted the accuracy of our model. But first we needed a baseline. Our baseline model has around 650,000 parameters and our accuracy results can be found in Figure P below.

```
==============================
VALIDATION RESULTS
==============================
Top-1 Accuracy: 39.62%
Top-5 Accuracy: 65.60%
Top-10 Accuracy: 75.99%
==============================
```

Figure P: *Baseline Model Accuracy*

Half-Model Size Results

Our half-size model had around 300,000 parameters. The accuracy and training information can be found below in Figure Q. The results here show that there is slightly less accuracy for our half-size model.

```
Half-Size Model Parameters: 0.30 Million

========================================
Training Half-Size ViT
========================================
Ep 5/30: Train Loss: 3.6744 | Val Acc: 20.34%
Ep 10/30: Train Loss: 3.2082 | Val Acc: 26.88%
Ep 15/30: Train Loss: 2.9222 | Val Acc: 31.02%
Ep 20/30: Train Loss: 2.7082 | Val Acc: 33.98%
Ep 25/30: Train Loss: 2.5660 | Val Acc: 35.03%
Ep 30/30: Train Loss: 2.5053 | Val Acc: 35.71%

Running Final Evaluation for Half-Size ViT...
------------------------------
FINAL RESULTS: Half-Size ViT
Top-1 Acc:  35.71%
Top-5 Acc:  62.79%
Top-10 Acc: 73.83%
------------------------------
```

Figure Q: *Half-Model Size Accuracy*

Double-Model Size Results

Our double-size model had around 1.25 million parameters. The accuracy and training information can be found below in Figure R. The results here show that there is slightly less accuracy for our double-size model.

```
Double-Size Model Parameters: 1.25 Million

========================================
Training Double-Size ViT
========================================
Ep 5/30: Train Loss: 3.6136 | Val Acc: 20.58%
Ep 10/30: Train Loss: 2.9459 | Val Acc: 29.51%
Ep 15/30: Train Loss: 2.4531 | Val Acc: 35.99%
Ep 20/30: Train Loss: 2.0535 | Val Acc: 37.74%
Ep 25/30: Train Loss: 1.7516 | Val Acc: 38.58%
Ep 30/30: Train Loss: 1.6302 | Val Acc: 38.59%

Running Final Evaluation for Double-Size ViT...
------------------------------
FINAL RESULTS: Double-Size ViT
Top-1 Acc:  38.59%
Top-5 Acc:  65.20%
Top-10 Acc: 75.62%
------------------------------
```

Figure R: *Double-Model Size Accuracy*

10x-Model Size Results

Our 10x-size model had around 5.43 million parameters. The accuracy and training information can be found below in Figure R. The results here show that there is more accuracy fall off for the 10x-model compared to even our half and double sized models. The overfitting here is extremely apparent after epoch 20.

```
Large Model Parameters: 5.43 Million

========================================
Training Large ViT (Tiny Config)
========================================
Ep 5/30: Train Loss: 3.9021 | Val Acc: 16.52%
Ep 10/30: Train Loss: 3.0499 | Val Acc: 28.12%
Ep 15/30: Train Loss: 2.2884 | Val Acc: 33.94%
Ep 20/30: Train Loss: 1.4168 | Val Acc: 35.01%
Ep 25/30: Train Loss: 0.7514 | Val Acc: 33.48%
Ep 30/30: Train Loss: 0.5374 | Val Acc: 33.20%

Running Final Evaluation for Large ViT (Tiny Config)...
------------------------------
FINAL RESULTS: Large ViT (Tiny Config)
Top-1 Acc:  33.20%
Top-5 Acc:  57.72%
Top-10 Acc: 67.81%
------------------------------
```

Figure S: *10x-Model Size Accuracy*

Overall Results

When comparing the different model sizes the results were a bit surprising. Since we know that the advantage of transformers is their scalability it was surprising to find that increasing the parameters was hurting our accuracy.

Our explanation for these unexpected results is that TinyImageNet is too small of a dataset for those larger parameter sizes. So when we increased the parameters we were just causing more overfitting. What might be more surprising is that we found the optimal parameter size on accident while trying to match our CNN model.

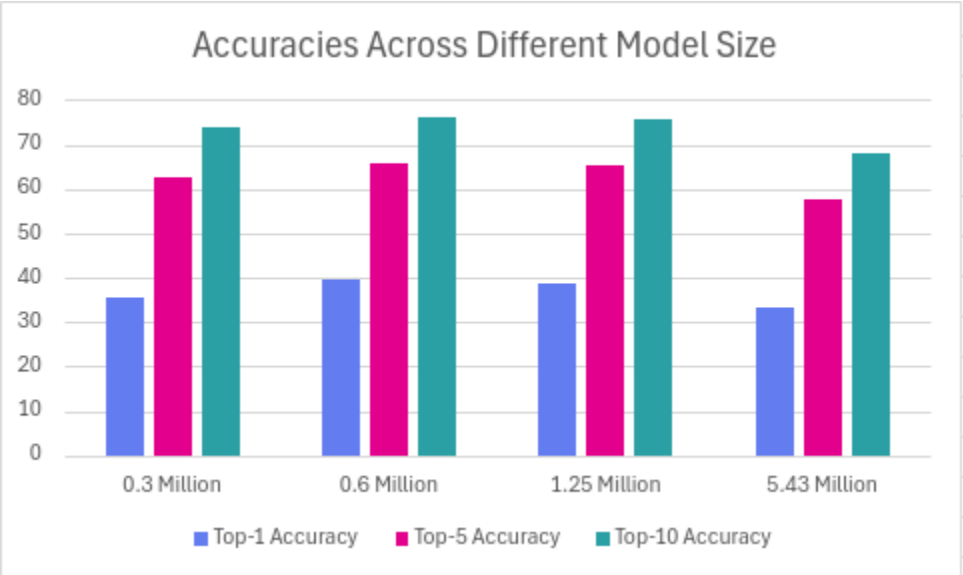


Figure T: *Model Size vs Accuracy Graph*

Stretch Goal 2 (Full Transformer HW Accelerator)

Refer to the Future Plans section of the document to see our potential plan on a full transformer hardware accelerator.

Stretch Goal 3 (C++ Optimizations CPU)

```
[Info]: --- Averages over 20 runs ---
[Info]: Avg L0: 0.533100ms
[Info]: Avg L1: 1.656500ms
[Info]: Avg L2: 1.642800ms
[Info]: Avg L3: 1.632500ms
[Info]: Avg L4: 1.628250ms
[Info]: Avg L5: 1.620800ms
[Info]: Avg L6: 1.629200ms
[Info]: Avg L7: 1.639900ms
[Info]: Avg L8: 1.632800ms
[Info]: Avg L9: 0.006250ms
[Info]: Avg L10: 0.000000ms
[Info]: Avg L11: 0.006100ms
[Info]: Avg L12: 0.000400ms
[Info]: Avg Full Inference: 13.963049ms
```

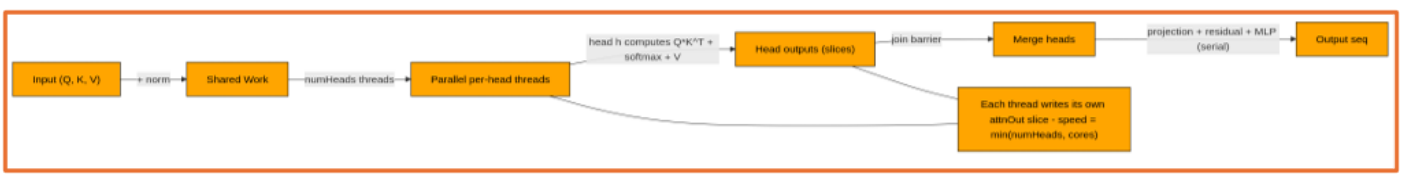
Naive

Naive path runs plain loops, LayerNorm computes mean/var per token, Linear does straightforward dot products, attention builds full Q/K/V, forms the score matrix, softmaxes each row, weights V, then applies projection + residual. MLP is two dense layers with GELU and a final residual. Every intermediate buffer is fully materialized (norm1Out, qkvOut, attnScores, attnOut, norm2Out, mlpHidden, mlpOut), so it's clear but bandwidth-heavy and cache-unfriendly

```
[Info]: --- Averages over 20 runs ---
[Info]: Avg L0: 0.508350ms
[Info]: Avg L1: 1.483100ms
[Info]: Avg L2: 1.482250ms
[Info]: Avg L3: 1.462300ms
[Info]: Avg L4: 1.468450ms
[Info]: Avg L5: 1.448550ms
[Info]: Avg L6: 1.519150ms
[Info]: Avg L7: 1.586200ms
[Info]: Avg L8: 1.660150ms
[Info]: Avg L9: 0.006900ms
[Info]: Avg L10: 0.000000ms
[Info]: Avg L11: 0.006400ms
[Info]: Avg L12: 0.000300ms
[Info]: Avg Full Inference: 12.759651ms
```

Multi-Threading

The threaded path parallelizes attention across many heads - after shared LayerNorm + QKV, it spawns one thread per head to compute that head's scores, softmax, and weighted sum into its own slice of attnOut, then joins and finishes projection, residuals, LayerNorm, and MLP on a single thread. Each head writes disjoint ranges

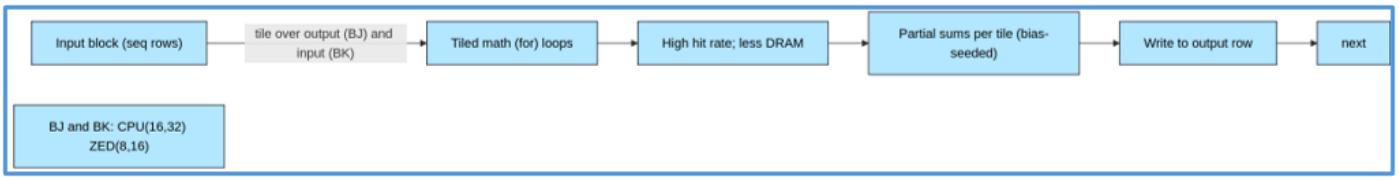



```
[Info]: --- Averages over 20 runs ---
[Info]: Avg L0: 0.542900ms
[Info]: Avg L1: 1.641150ms
[Info]: Avg L2: 1.635450ms
[Info]: Avg L3: 1.625050ms
[Info]: Avg L4: 1.617450ms
[Info]: Avg L5: 1.636150ms
[Info]: Avg L6: 1.627050ms
[Info]: Avg L7: 1.622650ms
[Info]: Avg L8: 1.619900ms
[Info]: Avg L9: 0.006400ms
[Info]: Avg L10: 0.000000ms
[Info]: Avg L11: 0.005900ms
[Info]: Avg L12: 0.000150ms
[Info]: Avg Full Inference: 13.586052ms
```

Tiling

Splitting big matrix operations into smaller blocks that fit in cache - process a tile of input rows by a tile of output channels (and a chunk of the inner dimension), accumulating partial sums before moving to the next tile. This cuts cache misses and memory bandwidth because each chunk of data is

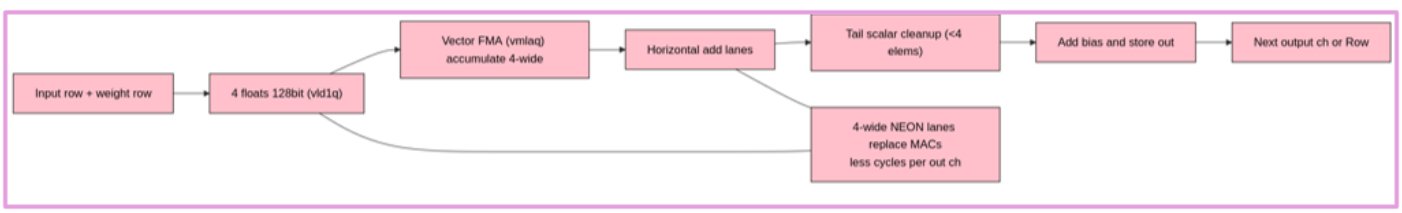
reused while it's hot instead of reloading entire matrices. Not Ideal for transformers as they only have 1 triple nested for loop of matmuls that already fits nicely into desired cache.




```
[Info]: --- Averages over 20 runs ---
[Info]: Avg L0: 0.527050ms
[Info]: Avg L1: 1.524350ms
[Info]: Avg L2: 1.508250ms
[Info]: Avg L3: 1.501000ms
[Info]: Avg L4: 1.496400ms
[Info]: Avg L5: 1.489850ms
[Info]: Avg L6: 1.497100ms
[Info]: Avg L7: 1.530400ms
[Info]: Avg L8: 1.500100ms
[Info]: Avg L9: 0.006500ms
[Info]: Avg L10: 0.000000ms
[Info]: Avg L11: 0.006200ms
[Info]: Avg L12: 0.000400ms
[Info]: Avg Full Inference: 12.701150ms
```

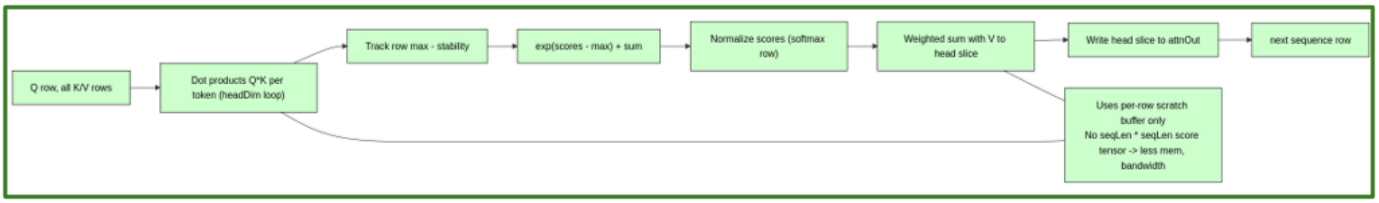
Attention Streaming

For each head, it processes one query token at a time: it computes that row's $Q \cdot K^T$ scores (scaled) into a small scores buffer, finds the row max, softmaxes in place, immediately forms the weighted sum over V, and writes the result to that head's slice in attnOut. The buffer is reused for the next row, never allocate a full [seqLen, seqLen] score matrix. Only just one row at a time cutting memory use and bandwidth.



SIMD (ARM NEON)

Our SIMD fast path is in when __ARM_NEON is defined, the dense matmul uses NEON intrinsics. For each output row/column, it loads inputs and weights with vld1q_f32, broadcasts zero with vdupq_n_f32, accumulates with vmlaq_f32, then horizontally sums the vector (vget_low/high, vadd_f32, vpadd_f32, vget_lane_f32), handles any tail scalars, and adds bias. Only runLinear is vectorized. Attention, softmax, and LayerNorm remain scalar.



Optimizations InActive

[Info]: --- Averages over 100 runs ---
[Info]: Avg L0: 17ms
[Info]: Avg L1: 231.839996ms
[Info]: Avg L2: 231.690002ms
[Info]: Avg L3: 231.869995ms
[Info]: Avg L4: 231.289993ms
[Info]: Avg L5: 231.500000ms
[Info]: Avg L6: 231.729996ms
[Info]: Avg L7: 231.690002ms
[Info]: Avg L8: 231.610001ms
[Info]: Avg Full Inference: 2158ms

SIMD (ARM NEON)

[Info]: --- Averages over 20 runs ---
[Info]: Avg L0: 17ms
[Info]: Avg L1: 231.ms
[Info]: Avg L2: 231.300003ms
[Info]: Avg L3: 231.449997ms
[Info]: Avg L4: 231.949997ms
[Info]: Avg L5: 230.899994ms
[Info]: Avg L6: 231.600006ms
[Info]: Avg L7: 231.350006ms
[Info]: Avg L8: 231.600006ms
[Info]: Avg Full Inference: 2144.649902ms

Challenges

Software

The first significant hurdle was just finding and training our model. We initially started with a github program that had pre-trained models that we could fine-tune to TinyImageNet. This program was too closed off and complicated for our purposes. We ended up landing on PyTorch – which was magnitudes superior compared to TensorFlow in our experience – as well as the *timm* Model library. This allowed us to set our hyperparameters, as well as tune it to our datasets. This library also allowed us to create and compare different sized models as well.

The second largest hurdle was modifying the framework and implementing our new/modified layers. Tyler did this part, and is not particularly skilled at C++ specifics – mostly generics – so some of the very well-written but confusing parts of the framework caused some roadbumps. In general, because the math had already been figured out in our python manual program, as well as already having model parameters, implementing the layers themselves went pretty smoothly all things considered. We had one issue left over from the CNN approach which was the full inference tested layer_11_output.bin. This was incorrect as we actually had 12 layer outputs. This did not take too long to fix, but given how late we were in the lab at this point specifically it did take longer to fix than we would like to admit.

A key challenge in Optimization was finding meaningful, readable spots to optimize inside this transformer's inference path. Most of it is already lean. We focused on the loops (attention and matmuls) and added streaming attention, cache-friendly tiling, and an optional NEON fast path, but deeper gains like training-time changes or larger architectural tweaks weren't options in the given timeframe.

Hardware

Overall we were satisfied with our hardware approach because we believe that it reduced the amount of debugging necessary. But there were still some struggles with making the FP32 MAC and then integrating it with software.

When creating the FP32 MAC the initial design was just to use a multiple symbol in behavior VHDL. This did not work for many reasons so after some research we found an IP block in Vivado that is made to do FP32 Multiplication. This was great because it was

more optimized and reduced the amount of hardware logic we needed to. But the downside was that we needed to figure out how to use the Vivado IP blocks inside of our VHDL. This overall was not difficult but definitely had a learning curve.

The other main challenge was integrating it in software. One issue we had was all the sums from the MACs were 0s. This was a software issue because we were sending the weights and inputs as packets without converting them to u32 first. This causes all of the sub zero float values to be rounded down to 0. We fixed this by adding helper functions that convert the float values to u32 and the u32 values back to floats when the MAC operation was completed.

Future Plans

Full Transformer Hardware Accelerator

This was our second stretch goal for our project but due to time constraints we were unable to implement this. But this goal was interesting to us and we were thinking about how to implement the accelerator during our entire project. Based on our occasional shower thoughts about the hardware accelerator this is how we would modify our Lab 6 Hardware Accelerator.

Hardware Accelerator in FP32

Our transformer model is in FP32 so we would like to keep it in that space to avoid quantization because it's scary. So the steps to convert our Int8 accelerator to FP32 would follow these. We would replace our Int8 MACs with our new FP32 MACs. This would allow us to do the FP32 MAC operations. Then we would remove the dequantization unit and replace it with a GeLU unit since our transformer uses GeLU instead of ReLU. Finally we would modify the output storage to increment by words instead of by bytes (FP32 takes 4 bytes instead of Int8 1 byte). With these steps we would be ready to do FP32 MAC operations and properly save them to BRAM.

Heads = Filters

The transformer model we have made has 8 heads which are independent of each other. So these could be parallelized. Since our accelerator is already built for doing MAC operations in groups of four this would work well since eight is divisible by four evenly. In software we would treat our heads like filters. An aspect to consider is if we would be able to fit the entire head inside of the filter block. We have not calculated if this would be possible or not.

Works Cited

Dosovitskiy, Alexey, et al. "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." *ArXiv:2010.11929 [Cs]*, 22 Oct. 2020, arxiv.org/abs/2010.11929.

Du, Dayou, et al. "Model Quantization and Hardware Acceleration for Vision Transformers: A Comprehensive Survey." *ArXiv.org*, 2024, arxiv.org/abs/2405.00314.

Hendrycks, Dan, and Kevin Gimpel. "Gaussian Error Linear Units (GELUs)." *ArXiv:1606.08415 [Cs]*, 8 July 2020, arxiv.org/abs/1606.08415.

Khalil, Mahmoud, et al. "A Comprehensive Study of Vision Transformers in Image Classification Tasks." *ArXiv.org*, 4 Dec. 2023, arxiv.org/abs/2312.01232.

Liu, Ze, et al. "Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows." *ArXiv:2103.14030 [Cs]*, 25 Mar. 2021, arxiv.org/abs/2103.14030.

Liu, Zhuang, et al. "A ConvNet for the 2020s." *ArXiv:2201.03545 [Cs]*, 10 Jan. 2022, arxiv.org/abs/2201.03545.

Raghu, Maithra, et al. "Do Vision Transformers See like Convolutional Neural Networks?" *ArXiv.org*, 3 Mar. 2022, arxiv.org/abs/2108.08810.

Touvron, Hugo, et al. “Training Data-Efficient Image Transformers & Distillation through Attention.” *ArXiv:2012.12877 [Cs]*, 15 Jan. 2021, arxiv.org/abs/2012.12877.

Vaswani, Ashish, et al. “Attention Is All You Need.” *Cornell University*, 12 June 2017, arxiv.org/abs/1706.03762.